Hyperlink / HyperFrames Video Generation Dashboard

Comprehensive System Architecture Audit & Engineering Report

Audit Date: October 16, 2026

Audited Target: Hyparlink Web Dashboard, Convex Backend, Ollima AI Engine, and HyperFrames Video Generation Pipeline

Workspace Root: `/Users/apple/Desktop/hyparlink dashbord`

Audit Purpose: Inspect, document, and report existing application architecture; verify frontend, Convex, Ollima, HyperFrames, and FFmpeg implementations; and design an architecture blueprint for adding a direct HTML/CSS/JavaScript video creation workflow that operates completely without Ollima.

---

1. Executive Summary

This architecture audit provides an exhaustive, evidence-based technical inspection of the **Hyparlink / HyperFrames Video Generation Dashboard**. The system is a full-stack, enterprise-grade video creation platform enabling users to produce broadcast-quality 1080p video compositions using modern web technologies (HTML, CSS, GSAP, SVG), autonomous AI planning, digital avatar synthesis, intelligent soundtrack scoring, and high-performance rendering.

Primary Audit Findings

1. **Frontend Architecture:** The user interface is built on **Next.js 14.2.7 (App Router)** and **React 18.3.1**, styled with **TailwindCSS** and **shadcn/ui** components. Core pages include the video creation studio (`create-video/page.tsx`), a two-tier live editor (`editor/[projectId]/page.tsx`) with embedded HyperFrames Studio, a workspace asset manager (`assets/page.tsx`), and real-time job monitors (`jobs/page.tsx`).

1. **Backend Architecture:** The application operates a **hybrid backend**:

- **Convex (v1.19.2):** Serves as the primary real-time database, authentication provider, multi-tenant workspace governor, and asynchronous job scheduler.
- **Node.js Echo Server (`backend-echo/server.mjs`):** Acts as the domain media engine, handling local process orchestration, file system project mapping, external AI completions, storage synchronization, and HyperFrames Studio sessions on dedicated ports.

1. **Ollima AI Integration:** Deeply integrated into `ai-composition-generator/src/ai/provider.ts` . It supports both native Ollima endpoints (`/api/chat`) and OpenAI-compatible endpoints (`/v1/chat/completions`) using structured JSON outputs (`format: "json"` or `response_format: { type: "json_object" }`). Prompts are orchestrated through `StoryboardPlanner` and `ScenePlanner` to generate structured scene graphs, GSAP animation directives, and creative briefs.

1. **HyperFrames Integration: Fully Verified and Real (Not a Mock or Placeholder).** The repository depends directly on `hyperframes@0.8.139` . Rendering is executed directly via `npx --yes hyperframes@0.8.139 render --resolution 1080p --format mp4` , compiling HTML/GSAP/DOM compositions into deterministic MP4 videos via headless browser rendering.

1. **FFmpeg & Audio Integration: Fully Implemented.** FFmpeg and FFprobe are dynamically resolved from the host environment. FFprobe conducts headless stream verification (codec, resolution, duration, bitrate, audio channels), while FFmpeg powers programmatic frame extraction, broadcast loudness normalization (`loudnorm=I=-16:TP=-1.5:LRA=11`), and speech-aware music ducking (`sidechaincompress`).

1. **Ollima Dependency Status: Currently, all new video generation requests initiated from the dashboard (`create-video/page.tsx`) unconditionally invoke Ollima.** However, the live editor (`editor/[projectId]/page.tsx` -> `POST /editor/project/:projectId/render`) already possesses a working, direct-to-renderer execution path that compiles user-edited HTML/CSS files without invoking the LLM.

1. **Feasibility of Direct-Code Workflow: Highly Feasible.** The underlying validation engine (`Validator.validate`), layout inspector (`DesignValidator.inspectAndRepair`), project scaffolder (`ProjectGenerator.createProject`), and renderer (`Renderer.render`) are already modular and completely decoupled from LLM execution. Adding a direct-code pipeline requires introducing an input mode switcher, a direct-code scaffolding endpoint, and modest schema extensions in Convex.

---

2. Existing Architecture Overview

The system architecture is structured across four primary operational tiers:

```
graph TB
    subgraph Client Tier [Client Tier - Browser]
        UI[Next.js 14 App Router]
        PromptInput[Prompt Bar & Config Controls]
        LiveEditor[Live Editor & Scene Inspector]
        StudioIframe[Official HyperFrames Studio IFrame]
    end

    subgraph Convex Tier [Convex Real-Time Cloud / Self-Hosted]
        CvxAuth[Auth & Workspaces]
        CvxDB[(Projects, Jobs, Assets Tables)]
        CvxScheduler[Task Scheduler]
        CvxWebhook[HTTP /backend/webhook]
    end

    subgraph Domain Engine Tier [Domain Media Backend :4000]
        EchoServer[backend-echo/server.mjs]
        StudioDaemon[HyperFrames Studio Daemon :3058+]
        AssetStore[(Storage Workspaces & CDN)]
    end

    subgraph Core Pipeline Tier [ai-composition-generator]
        Pipeline[VideoGenerationPipeline]
        AIProvider[AIProvider - Ollima Client]
        Synthesizer[CompositionSynthesizer]
        Validator[Validator & DesignValidator]
        HFRenderer[Renderer - hyperframes CLI]
        AudioMixer[AudioMixer - FFmpeg Engine]
    end

    UI -->|WebSocket Subscriptions & Mutations| CvxDB
    UI -->|HTTP POST /api/generate & /editor/*| EchoServer
    CvxScheduler -->|HTTP POST /jobs/run| EchoServer
    EchoServer -->|Dispatches Pipeline| Pipeline
    EchoServer -->|Spawns Studio Session| StudioDaemon
    StudioDaemon -.->|IFrame Embed| StudioIframe
    Pipeline -->|Prompt Synthesis| AIProvider
    AIProvider -->|OLLAMA_BASE_URL:11434| Ollima[Local/Remote Ollima LLM]
    Pipeline -->|Generates HTML/GSAP| Synthesizer
    Synthesizer -->|Validates AST & CSS| Validator
    Validator -->|Executes Render| HFRenderer
    HFRenderer -->|Executes npx hyperframes render| HFRenderer
    HFRenderer -->|Audio Ducking & FFprobe| AudioMixer
    Pipeline -->|Posts Results & Progress| CvxWebhook
    CvxWebhook -->|Patches Rows| CvxDB
```

---

3. Frontend Architecture

3.1 Framework, Version, and Runtime

- **Framework:** Next.js 14.2.7 (App Router).
- **Core Dependencies:** React 18.3.1, TypeScript 5.5.4, Bun 1.1.26 package manager, Turbo 2.1.1 monorepo orchestrator.
- **UI & Styling:** TailwindCSS, Geist Font (`geist`), Lucide React icons, Sonner (`sonner: 2.0.7`) toasts, TanStack Form, and Radix UI primitives (`@v1/ui`).
- **Client State & Routing:** `next-international` for localized routing (`/[locale]/...`), `nuqs` for URL search param state, and custom React Context (`WorkspaceProvider`).

3.2 File Structure & Key Components

Component / View	Source File Path	Architectural Responsibility
Root Dashboard Layout	`apps/app/src/app/[locale]/(dashboard)/layout.tsx`	Authenticated shell with multi-tenant sidebar, command palette, theme switcher, and notifications.
Create Video Studio	`apps/app/src/app/[locale]/(dashboard)/create-video/page.tsx`	Main video generation studio (3,943 lines) containing prompt input, mode toggles, duration, aspect ratio, avatar picker, and asset drawer.
Live Editor & Inspector	`apps/app/src/app/[locale]/(dashboard)/editor/[projectId]/page.tsx`	Split-view workspace (1,281 lines) embedding official HyperFrames Studio via iframe, companion visual inspector, and code editor.
Projects Overview	`apps/app/src/app/[locale]/(dashboard)/projects/page.tsx`	Real-time project catalog with status badges, thumbnail cards, and direct navigation.
Asset Library	`apps/app/src/app/[locale]/(dashboard)/assets/page.tsx`	Workspace asset management interface for image, video, audio, and logo files.
Jobs Monitor	`apps/app/src/app/[locale]/(dashboard)/jobs/page.tsx`	Live asynchronous job queue tracking progress, stages, and error states.
Workspace Provider	`apps/app/src/lib/workspace-provider.tsx`	React Context synchronizing the active workspace ID (`current`) across all pages.

3.3 Prompt Input and Mode Toggles

Located in `create-video/page.tsx` (Lines 2400–2650):

- **Prompt Textarea:** Expandable text area supporting natural language descriptions, preset templates (e.g., Rolex Luxury Ad, Karnataka Tourism, Vikas Swami PPT), and URL scraping directives.
- **Creation Modes:**

1. **Autopilot Mode:** Generates storyboard, assets, audio, and video in an autonomous single pass.
1. **Plan Mode (Video Agent):** Drafts a comprehensive structured production plan (scenes, narration script, visual direction, asset recommendations) for interactive user review before rendering.

- **Controls & Selectors:**
- Duration picker: 5s, 15s, 30s, 45s, 60s, 70s, 90s.
- Aspect Ratio: 16:9 Landscape (1920x1080), 9:16 Portrait (1080x1920), 1:1 Square (1080x1080).
- Avatar & Voice: HeyGen avatar picker modal (`sarah_studio` , `a9cde5fa...`) and neural voice selector (`71fbdde0...`).
- Soundtrack & Ducking: Automated soundtrack matcher, audio ducking toggle, and volume sliders.

3.4 Live Editor & Studio IFrame

Located in `editor/[projectId]/page.tsx` (Lines 160–235):

- The frontend calls `POST /editor/session/${projectId}` with the active workspace header.
- The backend initializes a dedicated, isolated official HyperFrames Studio server on an allocated port (e.g., `http://localhost:3058/#project/`).
- The page renders the Studio GUI within an iframe alongside a custom Scene Inspector that provides direct controls for headlines, subtitles, accent colors, and raw code edits.

---

4. Convex Backend Architecture

Convex operates as the primary reactive database and multi-tenant authorization layer.

4.1 Component Inventory

Component	File Path	Purpose	Current Behaviour
Schema Definition	`packages/backend/convex/schema.ts`	Database schema for 12 core tables.	Defines tables for `projects`, `jobs`, `assets`, `files`, `workspaces`, `members`, `users`, `auditLogs`, `usage`, `counters`, `notifications`, `apiKeys`.
Projects Module	`packages/backend/convex/projects.ts`	Video project lifecycle management.	Queries and mutations: `list`, `get`, `create`, `saveProductionPlan`, `approveAndRenderPlan`, `updateProjectResult`. Creates project record, dispatches async jobs.
Jobs Module	`packages/backend/convex/jobs.ts`	Asynchronous task queue & worker dispatcher.	Mutations: `create`, `setStatus`, `complete`. Action `processJob` executes HTTP POST to `${BACKEND_BASE_URL}/jobs/run`. On completion, patches project record.
HTTP Webhook Router	`packages/backend/convex/http.ts`	Inbound webhooks from backend engine.	Route `/backend/webhook` receives `job.progress` and `job.result`, updating Convex tables reactively without polling.
Organizations & Workspaces	`packages/backend/convex/orgs.ts`	Multi-tenant workspace governance.	Implements RBAC (`owner`, `admin`, `member`), workspace creation, membership checks (`requireMember`), and user defaults.
Assets Module	`packages/backend/convex/assets.ts`	Media library management.	CRUD operations for images, videos, audio tracks, and brand logos with storage key references.
File Storage	`packages/backend/convex/files.ts`	Metadata for files stored in Convex storage.	Interacts with Convex `_storage` tables for uploaded binary blobs.
Authentication	`packages/backend/convex/auth.ts`	User authentication provider.	Powered by `@convex-dev/auth` supporting passwordless OTP and OAuth providers.

4.2 How Generation Requests Flow Through Convex

1. The client invokes `api.projects.create`.
1. Convex verifies user workspace membership via `requireMember()`.
1. A row is inserted into `projects` (`status: "generating"`).
1. If in Autopilot mode, a corresponding row is inserted into `jobs` (`kind: "video.generate"`, `status: "pending"`).
1. Convex executes `ctx.scheduler.runAfter(0, internal.jobs.processJob, { jobId })`.
1. `processJob` issues an HTTP POST to `${BACKEND_BASE_URL}/jobs/run`.
1. The external backend posts progress updates back to Convex `/backend/webhook` (`type: "job.progress"`).
1. When rendering finishes, the backend posts `/backend/webhook` (`type: "job.result"`), which patches `projects` with `status: "completed"`, `outputUrl`, `storageKey`, `cdnUrl`, `fileSize`, and `renderTimeSeconds`.

---

5. Ollima Integration

5.1 Architecture & Implementation Details

- **Primary Client:** Implemented in `ai-composition-generator/src/ai/provider.ts` within the `AIProvider` class.
- **Configured Endpoint:** `process.env.OLLAMA_BASE_URL` or `process.env.AI_ENDPOINT`, defaulting to `http://localhost:11434`.
- **API Protocol:** Supports dual protocols:

1. **OpenAI Compatibility Mode (`/v1/chat/completions`):** Formats messages as `ChatCompletionMessageParam[]` and enforces structured JSON with `response_format: { type: "json_object" }`.
1. **Native Ollima Mode (`/api/chat`):** Formats request as `{ model, messages, format: "json", stream: false, options: { temperature } }`.

- **Supported Models:** Default fallback is `deepseek-v4-flash`, with support for `glm-5.3-flash`, `gemma4`, and other local or remote Ollima models.

5.2 Prompt Orchestration & Generation Pipeline

1. **Creative Brief Planning (`CreativeBriefPlanner.plan`):** Analyzes the prompt to derive mood, pacing, color palette, visual themes, and layout archetype.
1. **Storyboard Generation (`StoryboardPlanner.plan`):** Calls `AIProvider.complete()` with `OLLIMA_STORYBOARD_SYSTEM_PROMPT` (`prompts.ts`, Line 353).
1. **Structured Response:** The model outputs a strict JSON storyboard:

```json
```

{

"title": "Title of Video",

"targetDuration": 30,

"scenes": [

{

"id": "scene-1",

"duration": 6,

"headline": "Main Heading",

"subtitle": "Secondary description",

"visualElements": [...],

"animation": { "type": "kinetic_text", "easing": "power2.out" }

}

]

}

```

1. **Code Synthesis ( `CompositionSynthesizer.synthesize` ):** Converts the JSON storyboard into HyperFrames composition HTML files ( `index.html` , `compositions/scene_*.html` , `project.json` , `meta.json` ).

1. **Self-Repair Loop:** If syntax or validation errors are detected by `Validator.validate()` , `generate-video.ts` sends targeted error messages back to the LLM to patch the failing scene HTML (up to 3 attempts).

### 5.3 Error and Timeout Handling

- **Timeout:** 60-second AbortController on API requests.
- **HTTP 401/403 Handling:** Catches authentication failures and free-tier model restrictions.
- **HTTP 429 Handling:** Catches rate limits with explicit backoff messaging.
- **HTTP 404 Handling:** Catches missing models with clear instructions to pull the model.
- **Connection Errors:** Catches `ECONNREFUSED` and outputs diagnostic steps.

---

## 6. HyperFrames Integration

### 6.1 Integration Verification

**Audit Finding: HyperFrames is fully integrated and functional.**

- **Package Dependency:** `"hyperframes": "0.8.139"` is declared in `ai-composition-generator/package.json` and resolved in `node_modules` .
- **CLI Rendering:** Verified in `ai-composition-generator/src/hyperframes/renderer.ts` :

```bash

npx --yes hyperframes@0.8.139 render --resolution 1080p --format mp4 -o ""

```

- **Studio Daemon:** In `backend-echo/server.mjs` (`startStudioServerSession`), the server spawns:

```bash

npx --yes hyperframes@0.8.139 studio

```

listening on an isolated port, configured with an active project tunnel and iframe-compatible CSP headers.

6.2 Composition Specification

The generated compositions strictly adhere to the HyperFrames HTML specification:

- **Root Wrapper:** Declares
  .
- **Sub-compositions:** Uses .
- **Clips & Managed Media:** Media elements use `data-start` , `data-duration` , `data-media-start` , and `data-has-audio="true"` rather than imperative JavaScript `play()/pause()` calls.
- **Animations:** Powered by GSAP 3.12.5 targeting CSS transforms (`translate3d` , `scale` , `opacity`) without conflicting inline styles.
```

7. FFmpeg & Rendering Architecture

7.1 Executable Discovery

`ai-composition-generator/src/system/environment-checker.ts` verifies:

- `ffmpeg`: Detected at `/usr/local/bin/ffmpeg` or `/opt/homebrew/bin/ffmpeg`.
- `ffprobe`: Detected at `/usr/local/bin/ffprobe` or `/opt/homebrew/bin/ffprobe`.
- Supports H.264 video decoding/encoding and AAC audio multiplexing.

7.2 Media Processing Workflows

1. **Stream Verification (`Renderer.probeVideo`):** Runs `ffprobe` in JSON mode to verify stream dimensions (1920x1080), framerate (30.00 fps), duration, container format, and audio stream integrity.
1. **Programmatic Frame Extraction (`Renderer.extractInspectionFrames`):** Extracts 3 inspection frames (at 25%, 50%, and 75% of total duration) into `/inspect/frame_*.jpg` for automated headless QA without launching a browser.
1. **Soundtrack Multiplexing & Ducking (`AudioMixer.mixVideoAudio`):**

- Resamples audio streams to 48kHz stereo.
- Applies sidechain compression ducking (`sidechaincompress=threshold=0.08:ratio=8:attack=80:release=450`) to attenuate background music by ~14dB whenever narration is active.
- Applies ITU-R BS.1770 broadcast loudness normalization (`loudnorm=I=-16:TP=-1.5:LRA=11`).

8. Current End-to-End Video Generation Workflow

```
sequenceDiagram
    autonumber
    actor User as User
    participant Web as Next.js Frontend
    participant Cvx as Convex Backend
    participant Echo as Echo Server (:4000)
    participant Pipe as VideoGenerationPipeline
    participant Ollima as Ollima LLM (:11434)
    participant HF as HyperFrames CLI
    participant FFmpeg as FFmpeg / FFprobe
    participant Disk as File Storage / CDN

    User->>Web: Enter Prompt & Select Options (create-video)
    Web->>Cvx: api.projects.create({ prompt, duration, ... })
    Cvx->>Cvx: Insert projects row (status: "generating")
    Cvx->>Cvx: Insert jobs row (status: "pending")
    Cvx->>Echo: POST /jobs/run { jobId, projectId, input }
    Echo->>Pipe: VideoGenerationPipeline.run(prompt, options)
    Pipe->>Ollima: POST /api/chat (Creative brief & storyboard)
    Ollima-->>Pipe: JSON Storyboard & Scene Directives
    Pipe->>Pipe: Synthesize HTML, CSS, GSAP (CompositionSynthesizer)
    Pipe->>Pipe: Validate & Auto-Repair (DesignValidator)
    Pipe->>HF: npx hyperframes render --resolution 1080p -o video.mp4
    HF-->>Pipe: Rendered video.mp4
    Pipe->>FFmpeg: ffprobe stream check & frame extraction
    Pipe->>FFmpeg: AudioMixer sidechain ducking & soundtrack mix
    Pipe->>Disk: Save output MP4 & inspection frames
    Pipe->>Echo: Return PipelineReport
    Echo->>Cvx: POST /backend/webhook (type: "job.result")
    Cvx->>Cvx: Patch projects (status: "completed", outputUrl: ...)
    Cvx-->>Web: Real-time update via WebSocket query
    Web-->>User: Display completed video player & Live Editor link
```

Detailed Stage Breakdown

Stage	Subsystem	Function / File	Input Data	Output Data	Error Handling	Implementation Status
1. Input Submission	Frontend	`handleGenerateClick` `create-video/page.tsx`	Prompt string, duration, aspect ratio, avatar selection.	Project parameters object.	Validates non-empty prompt; shows user toast.	**VERIFIED**
2. Convex Job Creation	Convex	`projects:create` `convex/projects.ts`	Project parameters & workspace ID.	`projectId`, `jobId`.	Verifies membership; rolls back on auth failure.	**VERIFIED**
3. Dispatch to Echo	Convex Scheduler	`jobs:processJob` `convex/jobs.ts`	`jobId`.	HTTP request to `${BACKEND_BASE_URL}/jobs/run`.	Catches HTTP errors; sets job `error` state.	**VERIFIED**
4. AI Planning	AI Pipeline	`AIGenerator.generateProject` `generator.ts`	Prompt string, asset metadata, duration.	Structured JSON storyboard.	60s timeout abort; returns friendly model errors.	**VERIFIED**
5. Composition Synthesis	AI Pipeline	`CompositionSynthesizer` `composition-synthesizer.ts`	Storyboard JSON, themes, fonts.	Project files (`index.html`, `scene_*.html`).	Strict HTML escaping; fallback template generation.	**VERIFIED**
6. Scaffolding	HyperFrames	`ProjectGenerator.createProject` `project-generator.ts`	File payload, jobId.	Local project folder on disk.	Verifies directory creation; copies assets.	**VERIFIED**
7. Validation & Repair	HyperFrames	`DesignValidator.inspectAndRepair` `design-validator.ts`	Project folder path.	Validation report & patched HTML.	Auto-repairs missing IDs, transforms, and timing.	**VERIFIED**
8. Video Rendering	HyperFrames CLI	`Renderer.render` `renderer.ts`	Project folder, output MP4 path.	Uncompressed MP4 file.	600s timeout; preserves existing render on crash.	**VERIFIED**
9. Stream Verification	FFprobe	`Renderer.probeVideo` `renderer.ts`	Rendered MP4 path.	`ProbeMetadata` (fps, res, codec).	Logs warning; falls back to requested defaults.	**VERIFIED**
10. Audio Ducking	FFmpeg	`AudioMixer.mixVideoAudio` `audio-mixer.ts`	Video MP4, music track, speech audio.	Final multiplexed MP4.	Throws if inputs missing; handles single audio stream.	**VERIFIED**
11. Webhook Notification	Backend Echo	`sendWebhook` `server.mjs`	`jobId`, `projectId`, `result`.	HTTP POST to `/backend/webhook`.	Retries with backoff; logs failure.	**VERIFIED**
12. Client Display	Frontend	`useEffect` / `useQuery` `create-video/page.tsx`	Real-time project row update.	Active video player & download controls.	Displays error toast if job fails.	**VERIFIED**

9. Existing Database Schema and Job System

Convex database schema (`packages/backend/convex/schema.ts`):

```
erDiagram
    WORKSPACES ||--o{ PROJECTS : contains
    WORKSPACES ||--o{ JOBS : executes
    WORKSPACES ||--o{ ASSETS : stores
    WORKSPACES ||--o{ MEMBERS : enrolls
    USERS ||--o{ MEMBERS : belongs_to
    USERS ||--o{ PROJECTS : creates
    PROJECTS ||--o| JOBS : references

    WORKSPACES {
        id string PK
        string name
        id ownerId FK
        number createdAt
    }

    PROJECTS {
        id string PK
        id workspaceId FK
        id userId FK
        string name
        string prompt
        string status
        number duration
        number width
        number height
        number fps
        string hyperframesVersion
        string outputUrl
        string storageKey
        string mode
        any productionPlan
        number createdAt
    }

    JOBS {
        id string PK
        id workspaceId FK
        id userId FK
        string kind
        string status
        any input
        any result
        string error
        number createdAt
    }

    ASSETS {
        id string PK
        id workspaceId FK
        string name
        string type
        string storageProvider
        string storageKey
        string storageUrl
        string cdnUrl
    }
```

---

10. File Storage & Asset Management

The platform utilizes a **hybrid four-tier storage hierarchy**:

```
graph LR
    subgraph Storage Hierarchy
        T1[Tier 1: Convex Storage]
        T2[Tier 2: Cloudflare R2 / AWS S3]
        T3[Tier 3: Local File System]
        T4[Tier 4: CDN Edge Route]
    end

    T1 -->|Binary Blobs & Temp Files| Convex[(Convex _storage)]
    T2 -->|High-Def MP4 & Multi-Res Profiles| S3Bucket[(S3 / R2 Bucket)]
    T3 -->|HTML Compositions & Inspect Frames| Disk[(ai-composition-generator/storage)]
    T4 -->|Edge Delivery & Streaming| CDNUrl[https://.../cdn/workspaces/...]
```

1. **Convex File Storage (`_storage`):** Used for uploaded temporary attachments and dev-mode user assets.
1. **Cloud Object Storage (Cloudflare R2 / AWS S3):** Handled via `@aws-sdk/client-s3` . Rendered videos are uploaded to partitioned keys: `workspaces/{wsId}/projects/{projectId}/renders/{renderId}/video.mp4` .

1. **Local File System Storage:**

- Active compositions: `ai-composition-generator/generated-projects//`
- Render outputs: `ai-composition-generator/outputs/.mp4`
- Workspace media: `ai-composition-generator/storage/workspaces//assets/`

1. **Local HTTP CDN Provider:** `backend-echo/server.mjs` exposes `/storage/*`, `/cdn/*`, and `/outputs/*` with streaming range requests (`206 Partial Content`) and CORS headers.

---

11. Environment Configuration Overview

The application utilizes environment variables across `apps/app/.env.local`, `packages/backend/.env.local`, and root `.env`.

> [!IMPORTANT]
>
> In accordance with safety rules, no secret values or credentials are displayed below. Only variable names, scopes, and presence verifications are documented.

Variable Name	Defined Location	Scope / Purpose	Status in Repo
`NEXT_PUBLIC_CONVEX_URL`	`apps/app/.env.local`	Frontend Convex WebSocket URL	**CONFIGURED**
`NEXT_PUBLIC_BACKEND_URL`	`apps/app/.env.local`	Domain backend URL (`:4000`)	**CONFIGURED**
`NEXT_PUBLIC_STUDIO_URL`	`apps/app/.env.local`	HyperFrames Studio public tunnel base	**CONFIGURED**
`CONVEX_URL`	`packages/backend/.env.local`	Backend Convex RPC URL	**CONFIGURED**
`CONVEX_SELF_HOSTED_URL`	`packages/backend/.env.local`	Local Convex backend instance	**CONFIGURED**
`CONVEX_SELF_HOSTED_ADMIN_KEY`	`packages/backend/.env.local`	Admin key for self-hosted Convex	**CONFIGURED**
`BACKEND_BASE_URL`	Convex Environment	URL for Convex to call Echo Server	**CONFIGURED**
`BACKEND_WEBHOOK_SECRET`	Convex & Echo Environment	Webhook authentication secret	**CONFIGURED**
`OLLIMA_BASE_URL`	Root `.env` / Process Env	Ollima server endpoint	**CONFIGURED**
`AI_PROVIDER`	Root `.env` / Process Env	Provider selector (`ollima`)	**CONFIGURED**
`AI_MODEL`	Root `.env` / Process Env	Model selector (`deepseek-v4-flash`)	**CONFIGURED**
`HEYGEN_API_KEY`	Root `.env` / Process Env	Avatar generation API key	**CONFIGURED**
`S3_BUCKET` / `R2_BUCKET`	Root `.env`	Cloud object storage bucket	**OPTIONAL (Local fallback active)**

---

12. Identified Implementation Gaps

Before direct-code video generation can be implemented, the following architectural gaps must be addressed:

1. **Gap 1: Absence of Direct-Code Input Mode in UI:**

- In `create-video/page.tsx`, the interface only features a prompt textarea. Users cannot choose a "Direct Code" mode to supply `index.html`, `style.css`, and `animation.js`.

1. **Gap 2: Unconditional Ollima Invocation in Generation Pipeline:**

- `VideoGenerationPipeline.run()` (`generate-video.ts`) unconditionally executes `this.aiGenerator.generateProject(prompt)`. It has no conditional bypass to accept raw HTML/CSS/JS.

1. **Gap 3: Schema Lacks Source Code Storage Fields:**

- The Convex `projects` table (`schema.ts`) stores `prompt`, `duration`, `status`, etc., but has no fields for storing raw source code (`sourceHtml`, `sourceCss`, `sourceJs`, or `sourceFiles`).

1. **Gap 4: No Direct-Code Job Kind in Backend:**

- Convex `jobs` currently only processes `kind: "video.generate"` via LLM. A direct-code execution path (`kind: "video.render_direct"`) is required.

1. **Gap 5: Security Sandboxing for Untrusted Code:**

- Arbitrary user-supplied HTML/JS rendered via headless browser poses SSRF and infinite-loop risks. Security sandboxing must be formalized before executing untrusted client code.

---

13. Proposed Direct-Code Workflow

The target architecture introduces a dual-path pipeline that preserves the existing natural-language workflow while adding a direct-code rendering pipeline:

```
graph TD
    Input[User Input in Dashboard] --> Detector{Input Detector}

    %% Path 1: AI Prompt
    Detector -->|Natural Language Prompt| AIPath[AI Prompt Workflow]
    AIPath --> OllimaAPI[Ollama LLM API]
    OllamaAPI --> Storyboard[JSON Storyboard]
    Storyboard --> Synthesizer[Composition Synthesizer]
    Synthesizer --> CompFiles[Generated Project Files]

    %% Path 2: Direct Code (NO OLLIMA)
    Detector -->|Direct HTML/CSS/JS Code| CodePath[Direct Code Workflow — NO OLLIMA]
    CodePath --> ASTValidate[Sanitize & Validate Code]
    ASTValidate --> CodeScaffold[Direct Project Scaffolder]
    CodeScaffold --> CompFiles

    %% Shared Rendering Engine
    CompFiles --> HFValidator[HyperFrames DesignValidator]
    HFValidator --> HFRenderer[HyperFrames CLI Renderer]
    HFRenderer --> FFmpeg[FFmpeg Stream Verify & Audio Mix]
    FFmpeg --> Output[video.mp4 Output]
    Output --> Storage[(Cloud Storage & Convex Sync)]
    Storage --> Display[Display Final Video & Open in Studio]
```

---

14. Design of the Input Detection Approach

14.1 Evaluation of Approaches

Approach A: Explicit User-Selected Modes (Recommended)

- **UI Mechanism:** A prominent segmented mode switcher above the input area:
- `[ ✨ AI Prompt Mode ]` | `[ 💻 Direct Code Mode ]`
- **Advantages:**
- **100% Deterministic:** Zero probability of accidentally sending source code to Ollima.
- **Optimal UX:** Switches the editor component from a simple text area to a specialized code editor (Monaco or CodeMirror) with syntax highlighting, line numbers, and file tabs (`HTML` , `CSS` , `JS`).
- **Clear Intent:** The user explicitly signals their desired workflow.
- **Disadvantages:**
- Requires the user to select the appropriate tab manually.

Approach B: Automatic Heuristic Source-Code Detection

- **Mechanism:** Inspects input string with regex and AST heuristics:
- Checks for doctype declarations:
- Checks for root tags: `,`
  `, , ,`
